ft_transcendence

Live Auction Bidding Platform

Complete Team Technical Guide

What to build · What to learn · How to build it · When to build it

14 pts Minimum to pass	**18 pts** Your target (with buffer)	**10 days** Timeline to completion
5 people Team size	**9 tables** Database schema	**1 command** docker compose up

Generated: August 10, 2026 · 42 School — ft_transcendence · Pivot from POS to Auction Platform

This guide replaces the previous POS system plan. It builds directly on the Django backend skeleton already committed in the repository (accounts, products, auctions, orders apps).

Table of Contents

1. What Is an Online Auction Bidding Platform?
2. Project Requirements (from your brief)
3. Module Selection — 18 Points Plan
4. Tech Stack — What You Will Use and Why
5. What Every Team Member Must Learn
6. System Architecture — How Everything Connects
7. Database Schema — All 9 Tables Explained
8. Project Folder Structure
9. Docker Setup — Running with One Command
10. Auth System — Registration, Login, Roles, 2FA
11. Auction Lifecycle — Draft to Sold
12. Bidding Engine — Placing and Validating Bids
13. WebSocket System — Real-Time Bidding
14. Public API — Endpoints + Swagger
15. Internationalization — Arabic, English, French
16. Analytics Dashboard
17. 10-Day Execution Plan — Day-by-Day Breakdown
18. Team Roles and Responsibilities
19. Git Workflow and Commit Rules
20. Mandatory Checklist — Rejection Criteria
21. README.md Requirements

1. What Is an Online Auction Bidding Platform?

An online auction platform lets sellers list items for sale and lets buyers compete for them by placing increasing bids until a countdown ends. It is the software behind sites like eBay auctions or Catawiki: sellers submit items, an admin reviews them, bidders watch the price rise in real time, and whoever holds the highest bid when the clock hits zero wins and pays. Your project is a web-based auction platform: it runs in the browser, supports many bidders competing on the same item at the same time, and pushes live price updates the instant a new bid lands — no page refresh.

Core user flows your system must support

- Seller lists an item: Creates an auction event, adds one or more lots (items) with a starting price and a time window, and submits it for admin approval.
- Admin reviews listings: Approves or rejects submitted auction events with a reason, and schedules approved events to go live.
- Bidder competes live: Browses active lots, watches the current price and countdown, and places bids that update instantly for everyone else watching the same lot.
- Winner checks out: When a lot's timer ends, the highest bidder automatically gets an order for the final price, completes payment, and the seller ships the item.
- Everyone gets notified: Outbid alerts, auction-won alerts, and low-time warnings arrive in real time via WebSockets, not by refreshing the page.

2. Project Requirements (from your brief)

This is still the ft_transcendence final project of the 42 Common Core — only the product idea changed, from a Point of Sale system to a live auction bidding platform. You build as a team of 4, earn at least 14 module points, and submit a complete README. These mandatory requirements are unchanged by the pivot and apply regardless of what the product does:

- Web application with frontend, backend, and database
- Runs with a single command using Docker (docker compose up)
- Git used with clear commits from ALL team members
- Compatible with latest Google Chrome — zero console errors
- Privacy Policy page with real content, accessible from footer
- Terms of Service page with real content, accessible from footer
- Multiple users can be logged in and active simultaneously — this is not optional for an auction site, it is the whole point: several bidders must be able to bid on the same lot at once without conflicts
- All browser connections over HTTPS
- Responsive frontend (works on mobile and desktop)
- API keys / secrets stored in .env file — .env.example committed — .env gitignored
- Database with clear schema and well-defined relations
- Signup and login with hashed + salted passwords (Django's PBKDF2/bcrypt hasher)
- All forms validated on both frontend AND backend
- README.md in English with all required sections

Important note on the pivot: your Git repository already contains a working Django project skeleton (apps/accounts, apps/products, apps/auctions, apps/orders) plus Docker, PostgreSQL, and an Nginx config with HTTPS and a /ws/ route already wired up. You are not starting from zero — you are finishing what was already scaffolded, which is exactly why 10 days is realistic.

3. Module Selection — 18 Points Plan

You need 14 points minimum. We target 18 points for a comfortable 4-point safety buffer. Major modules = 2 points each, minor modules = 1 point each. Every module below maps naturally onto the auction domain — nothing here is bolted on for points.

Major modules — 12 points

Module	How it applies here
Web: Framework (Django + React)	Django REST Framework backend, React frontend. Full-stack framework combo = 2 points.
Web: Real-time features (Django Channels)	WebSocket consumers broadcast every bid instantly to everyone watching a lot.
User management: Standard auth	Profile, avatar, online status, bidder/seller activity history.
User management: Advanced permissions	USER vs ADMIN roles; admin approval workflow; object-level checks (a seller can't bid on their own lot).
Data: Analytics dashboard	Interactive charts: revenue, bids per hour, top lots, active bidders. Real-time updates.
Web: Public API	Secured API key, rate limiting, Swagger docs (drf-spectacular), 5+ endpoints (GET/POST/PUT/DELETE).

Minor modules — 6 points

Module	How it applies here
Web: ORM for database	Django's built-in ORM. Typed models, migrations, safe queries — already in use.
Web: Notification system	Toast notifications for outbid alerts, auction won, low-stock-style lot closing warnings.
User management: 2FA	TOTP two-factor authentication via an authenticator app (pyotp).
Accessibility: Multi-language (3 langs)	English, Arabic, French. i18next on the frontend, Django i18n on the backend.
Accessibility: RTL support	Arabic requires right-to-left layout. Full layout mirroring.
Data: Export/Import	Export bid history and orders to CSV. Import lots in bulk from a CSV file.

Total: 6 major (12 pts) + 6 minor (6 pts) = 18 points. ✓

4. Tech Stack — What You Will Use and Why

This is the real stack already reflected in your repository: a Django + Django REST Framework backend (not Node/Express), PostgreSQL, Docker, and Nginx with HTTPS already configured. The frontend is React + TypeScript, kept from the original plan since it is framework-agnostic to the backend choice. The one new addition is Django Channels + Redis, needed because auctions live and die by real-time bid updates.

Frontend

Technology	What it does
React 18	The UI framework. Components = reusable pieces of UI. State = data that changes.
TypeScript	JavaScript with types. Catches bugs before you run the code. Required.
Tailwind CSS	Utility-first CSS. Write class names instead of CSS files. Fast to learn.
React Router v6	Controls which page shows based on the URL. Handles protected routes.
Zustand	Tiny state management. Stores login state and the active bid draft globally.
React Query	Manages all API calls. Handles loading, error, caching automatically.
Native WebSocket / socket wrapper	Connects to the Django Channels endpoint for live bid updates and countdowns.
Recharts	Chart library for the analytics dashboard. Bar, line, pie charts.
i18next	Internationalization. Switches all text between English, Arabic, French.
Axios	Makes HTTP requests to your backend. Simpler than fetch().

Backend

Technology	What it does
Python 3.12	The language the whole backend is written in. Already the base image in your Dockerfile.
Django 6	Web framework. Already installed — handles models, admin, auth, migrations.
Django REST Framework (DRF)	Turns your Django models into a JSON API: serializers, viewsets, permissions.
Django Channels	Adds WebSocket support to Django. Powers the live bidding feed on top of the existing ASGI app.
Redis	In-memory store used as the Channels layer (fan-out of bid events) and for caching hot lot prices.
PostgreSQL	The database. Already configured in docker-compose.yml and settings.py.
djangorestframework-simplejwt	Issues signed JWTs for authentication, delivered as httpOnly cookies.
Django's password hasher	Hashes passwords with PBKDF2 by default. Never store plain text passwords.
DRF Serializers	Validates and shapes incoming/outgoing JSON. Backend-side form validation.
pyotp + qrcode	Generates and verifies TOTP codes for 2FA (like Google Authenticator).
drf-spectacular	Auto-generates OpenAPI/Swagger documentation from your DRF viewsets.
django-ratelimit / DRF throttling	Prevents abuse of your Public API endpoints.
Celery + Celery Beat (or a scheduled management command)	Closes lots automatically when their end_time passes and creates the winning order.

DevOps

Technology	What it does
Docker	Packages your entire app into containers. Runs anywhere.
Docker Compose	Orchestrates the containers (db, redis, backend, websocket, frontend, nginx) with one command.
Nginx	Web server that handles HTTPS and routes /api/ to the backend and /ws/ to the Channels process — already scaffolded in nginx/nginx.conf.
mkcert / provided self-signed cert	SSL certificates for HTTPS. Your repo already has nginx/ssl/server.crt and server.key.
GitHub	Version control. Every team member commits. Issues track tasks.

5. What Every Team Member Must Learn

The team's existing backend code is already Python/Django, not Node/Express — so the learning path below reflects that reality, not the original plan. This is a tighter, 10-day schedule, so treat Days 1–2 as a hard deadline for the fundamentals below, not a leisurely ramp-up.

JavaScript Fundamentals (Frontend pair — Day 1)

Topic	What to know
Variables & types	let, const, string, number, boolean, array, object.
Functions	Arrow functions, async/await. Most API calls are async.
Promises	await pauses until a value resolves. try/catch handles errors.
Array methods	.map(), .filter(), .reduce(), .find(). Used constantly for lot lists and bid feeds.
Destructuring	const { name, email } = user. Shortcuts for extracting values.
Modules	import / export. Every file is a module.
Template literals	`Current bid: $${amount}` — string interpolation with backticks.

Python Fundamentals (Backend pair — Day 1, if new to Python)

Topic	What to know
Variables & types	Python is dynamically typed but Django models declare field types explicitly.
Functions & decorators	def, @api_view, @permission_classes — decorators wrap views with extra behavior.
Classes	Django models, serializers, and viewsets are all Python classes you subclass.
List/dict comprehensions	[bid.amount for bid in lot.bids.all()] — Python's equivalent of .map().
Exceptions	try/except around anything that touches the database or an external service.
Virtual environments	Isolates project dependencies. Already scaffolded — see doc/setup.md.
f-strings	f"Lot {lot.lot_number}: {lot.product.name}" — already used throughout the existing models.py files.

Django + DRF (Backend pair — Day 1–2)

Topic	What to know
Models	Python classes that map to database tables. Already written for accounts, products, auctions, orders.
Migrations	python manage.py makemigrations / migrate. Generates and applies schema changes.
QuerySets	Product.objects.filter(available=True) — Django's ORM query language.
Serializers (DRF)	Convert model instances to/from JSON. Also where field-level validation lives.
ViewSets & routers	DRF classes that turn a serializer + queryset into full CRUD endpoints automatically.
Permission classes	IsAuthenticated, IsAdminUser, and custom classes like IsAuctionOwner.

Topic	What to know
Signals / services layer	Keep business logic (bid validation, lot closing) in services.py, not in views.
Django admin	Free CRUD UI at /admin/ — useful for seeding and debugging during development.

Django Channels (Real-time owner — Day 5–6, before building the bidding socket)

Topic	What to know
What Channels adds	Lets Django handle long-lived WebSocket connections, not just request/response HTTP.
Consumers	The WebSocket equivalent of a view. Handles connect, receive, disconnect.
Groups	A named channel, e.g. lot_42. Every browser watching lot 42 joins that group.
Channel layer (Redis)	The backend that lets any process broadcast a message to everyone in a group.
ASGI vs WSGI	ASGI supports async and WebSockets; WSGI is classic request/response only. Channels needs ASGI.

React (Frontend pair — Day 1–2)

Topic	What to know
Components	A component is a function that returns JSX. function BidPanel() { return ... }
Props	Data passed into a component. function LotCard({ lot }: Props).
useState	const [amount, setAmount] = useState(0). Reactive state — UI re-renders on change.
useEffect	Runs code after render. Used to open the WebSocket connection on mount and clean it up on unmount.
Event handling	onClick, onChange, onSubmit. Always e.preventDefault() in bid forms.
Conditional rendering	{isEnded ? <SoldBadge /> : <CountdownTimer end={lot.endTime} />}
Lists	{lots.map(lot => <LotCard key={lot.id} lot={lot} />)} — always a unique key.

Docker (Tech Lead — Day 1)

Topic	What to know
What Docker is	Packages your app and its dependencies into a container. 'Works on my machine' problem solved.
Dockerfile	Instructions to build an image. Your backend/Dockerfile already exists — extend it as needed.
docker-compose.yml	Defines multiple services and how they connect. Your project needs db, redis, backend, websocket, frontend, nginx.
Volumes	Persistent storage. postgres_data already keeps your database data across restarts.
Networks	Containers reach each other by service name, e.g. http://backend:8000, redis://redis:6379.

Topic	What to know
Commands to know	docker compose up --build, docker compose down, docker compose logs backend.

Git Workflow (ALL members — Day 1)

Topic	What to know
Basic commands	git clone, git add ., git commit -m "message", git push, git pull.
Branching	git checkout -b feature/bidding-engine. Work on a branch, never directly on main.
Commit messages	Present tense, specific: "Add minimum-increment validation to bid service", not "changes".
Pull Requests	Push your branch, open a PR on GitHub, another team member reviews, then merge.
Conflicts	git pull, resolve conflicts manually, git add, git commit.

Recommended free resources: MDN Web Docs (mdn.io) for JavaScript, React docs (react.dev), Django docs (docs.djangoproject.com), Django REST Framework docs (django-rest-framework.org), Channels docs (channels.readthedocs.io). The official docs are always the best source.

6. System Architecture — How Everything Connects

Your system has six layers, one more than a typical CRUD app, because real-time bidding needs a separate WebSocket process alongside the normal REST API. Understanding this flow is essential before writing any code — it is also exactly why nginx.conf in your repo already has two proxy_pass blocks.

Layer	Responsibility
Browser (React)	The user interface. Runs entirely in the user's browser. Makes HTTP requests via Axios. Opens a native WebSocket connection per lot page for live bids. Manages local state with Zustand.
Nginx (Reverse Proxy)	Receives all incoming connections on port 443 (HTTPS). Terminates SSL using the certs already in nginx/ssl/. Routes /api/* to the backend REST process. Routes /ws/* to the Channels (ASGI) process. Routes /* to the React static build.
Django + DRF (REST process)	Handles all normal HTTP requests: auth, auction event CRUD, lot CRUD, orders, payments, analytics. Validates data with DRF serializers. Checks JWT auth and role permissions. Calls the ORM to read/write Postgres.
Django Channels (ASGI process)	A second entry point into the same codebase, running an ASGI server. Handles only WebSocket connections. Consumers join a lot_{id} group per open lot page and broadcast new bids, outbid alerts, and lot-closed events to everyone in that group.
Redis	The channel layer. When the REST process (or the lot-closing task) needs to push an event to browsers, it publishes to Redis; Channels fans it out to every subscriber in the right group. Also usable as a cache for hot lot prices.
PostgreSQL	Stores all data permanently. Tables: users, categories, products, inventory, auction_events, auction_lots, bids, orders, payments. The ORM handles all SQL — you never write raw SQL.

Request example: a bid is placed

Bidder clicks "Place Bid" on lot #7 → Browser sends POST /api/auctions/lots/7/bids with the JWT cookie and { amount: 250 } → Nginx forwards to the REST process on :8000 → JWT middleware verifies the cookie → Serializer validates the payload → Bid service checks the lot is ACTIVE, the amount clears the current price plus the minimum increment, and the bidder isn't the seller → Bid saved, AuctionLot.highest_bid updated inside a DB transaction → REST process publishes a bid:new event to Redis on the lot_7 channel group → Every open browser tab subscribed to lot 7 (via the Channels process on :8080) receives the event over its WebSocket instantly → Their UI updates the current price and countdown without a refresh → The REST response returns the bid confirmation to the bidder who submitted it.

7. Database Schema — All 9 Tables Explained

This schema is already largely written in your repository (apps/accounts, apps/products, apps/auctions, apps/orders). This section documents it as the team's single source of truth — review it together and agree on any remaining gaps (like the 2FA fields on users) before writing more application code.

Table: users (accounts.CustomUser)

Field	Type	Purpose
id	UUID/int	Primary key — unique identifier for every user
email	String @unique	Login identifier — USERNAME_FIELD, must be unique
username	String	Display name / required field alongside email
password	String (hashed)	Django's built-in hasher — NEVER store plain text
phone_number	String, blank	Optional contact number
role	Enum: USER | ADMIN	Controls approval and moderation access throughout
is_staff / is_superuser	Boolean	Django admin access flags, set on superusers only
totp_secret / totp_enabled	String? / Boolean	Add these fields for the 2FA minor module
date_joined	DateTime	When the account was created — set automatically

Table: categories

Field	Type	Purpose
id	UUID/int	Primary key
name	String @unique	e.g. Watches, Art, Electronics, Collectibles

Table: products

Field	Type	Purpose
id	UUID/int	Primary key
name	String	Item display name
price	Decimal(10,2)	Reference / catalog price — not the auction price
description	Text, blank	Full item description shown on the lot page
category	FK → categories	Which category this item belongs to
available	Boolean	Soft delete / hide flag
created_at / updated_at	DateTime	Timestamps

Table: inventory

Field	Type	Purpose
id	UUID/int	Primary key
product	OneToOne → products	Each product has exactly one inventory record
quantity	Int	Units the seller has available to list as lots
reserved_quantity	Int	Units already tied up in an active lot

Table: auction_events

Field	Type	Purpose
id	UUID/int	Primary key
title / description	String / Text	What the auction event is called and about
organizer	FK → users	The seller running this auction event
status	Enum	DRAFT | PENDING_APPROVAL | REJECTED | APPROVED | SCHEDULED | ACTIVE | ENDED | CANCELLED
start_time / end_time	DateTime	The event's overall live window
submitted_at	DateTime?	When the organizer submitted for review
reviewed_by	FK → users, null	The admin who approved/rejected it
reviewed_at / rejection_reason	DateTime? / Text	When it was reviewed and why, if rejected

Table: auction_lots

Field	Type	Purpose
id	UUID/int	Primary key
auction_event	FK → auction_events	Which event this lot belongs to
product	FK → products	The item being sold in this lot
lot_number	PositiveInt	Unique within its event (unique_lot_number_per_event constraint)
starting_price	Decimal(10,2)	Opening price before any bids
highest_bid	FK → bids, null	Points at the current winning bid, if any
status	Enum	SCHEDULED | ACTIVE | CANCELLED | SOLD | UNSOLD
start_time / end_time	DateTime	This specific lot's countdown window

Table: bids

Field	Type	Purpose
id	UUID/int	Primary key
auction_lot	FK → auction_lots	Which lot this bid was placed on

Field	Type	Purpose
bidder	FK → users, null	Who placed the bid
amount	Decimal(10,2)	The bid amount
status	Enum: ACCEPTED | REJECTED	Whether the bid engine accepted it
rejection_reason	Enum, null	LOWER_THAN_CURRENT | AUCTION_NOT_ACTIVE | SELLER_BIDDING | BLOCKED_USER | MINIMUM_INCREMENT_NOT_MET
created_at	DateTime	Ordered newest-first for the live feed

Table: orders

Field	Type	Purpose
id	UUID/int	Primary key
auction_lot	OneToOne → auction_lots	The lot this order was generated from
buyer	FK → users	The winning bidder
seller	FK → users	The organizer of the auction event
final_price	Decimal(10,2)	The winning bid amount
shipping_address	Text	Where the item ships to
status	Enum	AWAITING_PAYMENT | PAID | PROCESSING | SHIPPED | DELIVERED | CANCELLED
created_at / updated_at	DateTime	Timestamps

Table: payments

Field	Type	Purpose
id	UUID/int	Primary key
order	FK → orders	Which order this payment attempt is for
transaction_reference	String @unique, null	Reference from the payment processor
amount	Decimal(10,2)	Amount charged
status	Enum	PENDING | SUCCESSFUL | FAILED
method	Enum	CARD (extend later)
failure_reason	Text, blank	Why a failed payment failed
created_at / completed_at	DateTime	Timestamps

8. Project Folder Structure

The backend structure already exists — do not reorganize it, extend it. The frontend does not exist yet and needs to be created from scratch by Frontend 1 and Frontend 2 on Day 1.

Backend (existing — extend, don't restructure)

Path	What goes here
backend/	Django project root — already exists
backend/manage.py	Django CLI entry point — already exists
backend/requirements.txt	Add djangorestframework, channels, channels-redis, djangorestframework-simplejwt, drf-spectacular, pyotp, qrcode, celery, django-cors-headers
backend/config/settings.py	Add INSTALLED_APPS entries for DRF, channels, corsheaders; add ASGI_APPLICATION, CHANNEL_LAYERS
backend/config/asgi.py	Already exists — extend to route HTTP to Django and /ws/ to Channels routing
backend/apps/accounts/	Already has CustomUser model. Add: serializers.py, views.py (register/login/me/2fa), permissions.py
backend/apps/products/	Already has Category, Product, Inventory. Add: serializers.py, views.py, CSV import endpoint
backend/apps/auctions/	Already has AuctionEvent, AuctionLot, Bid. Add: serializers.py, services.py (bid validation), consumers.py, routing.py
backend/apps/orders/	Already has Order, Payment. Add: serializers.py, views.py, tasks.py (auto-close lots → create order)
backend/apps/common/	Shared utilities: pagination classes, API key middleware, throttling, audit log helper

Frontend (new)

Path	What goes here
frontend/	New — React application, does not exist yet in the repo
frontend/package.json	Frontend dependencies
frontend/tailwind.config.ts	Tailwind CSS configuration
frontend/Dockerfile	Build React → static files, served by Nginx
frontend/public/locales/	Translation JSON files (en/, ar/, fr/)
frontend/src/api/	All Axios API call functions
frontend/src/sockets/useAuctionSocket.ts	Hook that opens/closes the WebSocket per lot page
frontend/src/components/ui/	Reusable UI: Button, Input, Modal, Badge, CountdownTimer
frontend/src/pages/Login.tsx / Register.tsx	Auth pages
frontend/src/pages/Browse.tsx	Public list of live and upcoming lots

Path	What goes here
frontend/src/pages/LotDetail.tsx	Live bidding page for a single lot
frontend/src/pages/MyAuctions.tsx	Seller's auction events (create, submit, view status)
frontend/src/pages/AdminApprovals.tsx	Admin queue to approve/reject submitted events
frontend/src/pages/MyBids.tsx / MyWins.tsx / Orders.tsx	Bidder's bid history, wins, and order status
frontend/src/pages/Analytics.tsx	Charts and reports
frontend/src/store/auth.store.ts	Zustand: current user state
frontend/src/store/bid.store.ts	Zustand: draft bid amount, live lot state
frontend/src/i18n.ts	i18next configuration
frontend/src/router.tsx	All routes with auth/role guards

9. Docker Setup — Running with One Command

docker compose up --build is the only command needed to run the entire system. Your repo already has db, backend, and nginx services with HTTPS working. This section adds the three pieces the auction pivot requires: redis (the Channels layer), a websocket service running the same codebase on an ASGI server, and the frontend.

Why two backend services?

Your existing nginx.conf already proxies /api/ to backend:8000 and /ws/ to backend:8080 — it was scaffolded with this split in mind. The cleanest way to satisfy that is two containers built from the same ./backend image: one runs a normal WSGI server (gunicorn) for REST traffic, the other runs Daphne (ASGI) for WebSocket traffic. Both share the same database and the same Redis channel layer, so a bid saved by the REST process is instantly broadcastable by the websocket process.

docker-compose.yml (target state)

```
services:
  db:
    image: postgres:16
    env_file: [.env]
    volumes: [postgres_data:/var/lib/postgresql/data]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
      interval: 5s
      retries: 5

  redis:
    image: redis:7-alpine
    restart: unless-stopped

  backend:
    build: ./backend
    env_file: [.env]
    command: gunicorn config.wsgi:application -b 0.0.0.0:8000
    depends_on: [db, redis]

  websocket:
    build: ./backend
    env_file: [.env]
    command: daphne -b 0.0.0.0 -p 8080 config.asgi:application
    depends_on: [db, redis]

  frontend:
    build: ./frontend
    volumes: [frontend_dist:/app/dist]

  nginx:
    build: ./nginx
    ports: ["443:443"]
    depends_on: [backend, websocket, frontend]
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf
      - ./nginx/ssl:/etc/nginx/ssl
      - frontend_dist:/usr/share/nginx/html:ro

volumes:
  postgres_data:
  frontend_dist:
```

.env additions

Add these on top of the existing DJANGO_SECRET_KEY / POSTGRES_* variables already in your .env.example:

```
REDIS_URL=redis://redis:6379/0
CHANNEL_LAYER_HOST=redis
CHANNEL_LAYER_PORT=6379
JWT_SECRET_KEY=REPLACE_WITH_64_CHARACTER_RANDOM_STRING
PUBLIC_API_KEY=REPLACE_WITH_RANDOM_API_KEY
MIN_BID_INCREMENT=5.00
```

10. Auth System — Registration, Login, Roles, 2FA

Authentication is the foundation everything else depends on. Build this first and get it fully working before touching the bidding engine. Backend 1 owns this, since it sits alongside the Docker/infra work they already own.

How JWT authentication works here

djangorestframework-simplejwt issues a signed access token when a user logs in, containing their user ID and role. The token is sent back as an httpOnly cookie rather than a response body field, so client-side JavaScript can never read it — the browser just attaches it automatically on every request. The backend verifies the signature and knows who is calling without hitting the database on every request.

Why httpOnly cookies instead of localStorage?

localStorage can be read by any JavaScript running on the page — including malicious scripts injected via XSS. An httpOnly cookie cannot be read by JavaScript at all; only the browser sends it, and only to your server. This is the industry-standard approach and matches what the original plan specified.

Auth API endpoints

Method	Endpoint	Access	Description
POST	/api/accounts/register	Public	Create new account. Body: { email, username, password }
POST	/api/accounts/login	Public	Login. Body: { email, password }. Sets httpOnly JWT cookie.
POST	/api/accounts/logout	Auth required	Clears the JWT cookie.
GET	/api/accounts/me	Auth required	Returns current user object (no password hash).
POST	/api/accounts/2fa/setup	Auth required	Generates TOTP secret. Returns QR code URL.
POST	/api/accounts/2fa/verify	Auth required	Verifies 6-digit code and enables 2FA.
POST	/api/accounts/2fa/confirm	Auth required	Completes login when 2FA is enabled.

Role-based access — who can do what

The CustomUser model already ships with a role field of USER or ADMIN — there is no separate 'seller' or 'bidder' role, because any user can list items and bid on other people's lots. Enforcement is a mix of role checks (USER vs ADMIN) and object-level ownership checks (only the organizer can edit their own event; a bidder can never bid on their own lot).

Feature	USER	ADMIN
Browse live/upcoming lots	✓	✓
Create & submit an auction event (become organizer)	✓	✓
Approve / reject a submitted auction event	✗	✓
Add lots to your own approved event	✓(own only)	✓
Place bids on lots you don't own	✓	✓

Feature	USER	ADMIN
View your own bid / order history	✓	✓
View all orders, bids, and the audit log	✗	✓
Manage users & roles, block a user from bidding	✗	✓
View analytics for your own auctions	✓	✓(global)
Access the Public API	✓(with key)	✓(with key)

11. Auction Lifecycle — Draft to Sold

This is the backbone of the whole product. Every other feature — bidding, WebSockets, orders, analytics — hangs off these two state machines, both of which are already modeled in apps/auctions/models.py. Nobody should write bidding code until this flow is agreed on as a team.

AuctionEvent status flow

Status	Meaning
DRAFT	Organizer is still editing. Not visible to anyone else.
PENDING_APPROVAL	Organizer submitted it. Sits in the admin's approval queue.
REJECTED	Admin rejected it with a rejection_reason. Organizer can edit and resubmit.
APPROVED	Admin approved it. Not yet visible to bidders until scheduled.
SCHEDULED	Approved and waiting for its start_time.
ACTIVE	Live — bidders can see and bid on its lots.
ENDED	start_time/end_time window has passed. Lots have been resolved (SOLD/UNSOLD).
CANCELLED	Organizer or admin pulled the event before it went live.

AuctionLot status flow

Status	Meaning
SCHEDULED	Belongs to an approved event; waiting for its own start_time.
ACTIVE	Bidders can place bids right now.
CANCELLED	Pulled before it started.
SOLD	end_time passed with at least one accepted bid. An Order was created for the highest bidder.
UNSOLD	end_time passed with zero bids. No Order is created.

The organizer_can_edit and editing_deadline properties

These already exist on AuctionEvent: editing_deadline is start_time minus one day, and organizer_can_edit is true only while the event isn't ACTIVE/ENDED/CANCELLED and the deadline hasn't passed. Use these in the serializer/view layer to reject edit requests server-side — never trust the frontend to hide the edit button and call that enough.

Closing a lot automatically

A lot doesn't close because someone happens to load the page after end_time — it closes because a scheduled job checks for it. Use Celery Beat (or, if the team wants to keep dependencies minimal for a 10-day sprint, a simple management command run in a loop inside its own lightweight container) that runs every few seconds and: finds ACTIVE lots whose end_time has passed, marks them SOLD or UNSOLD, creates the Order for the winner if SOLD, and broadcasts a lot:ended WebSocket event to that lot's group. This must be idempotent — running it twice on the same lot must not create two orders.

12. Bidding Engine — Placing and Validating Bids

The Bid model already has status (ACCEPTED/REJECTED) and a RejectionReason enum with exactly the five reasons below — this tells you precisely what the validation service needs to check, in order. Person 3 owns this service; it is the single most important piece of business logic in the project because it protects real money.

Validation order (stop at the first failure)

Check	Rejection reason if it fails
Lot must be ACTIVE	AUCTION_NOT_ACTIVE — reject bids on SCHEDULED, CANCELLED, SOLD, or UNSOLD lots.
Amount must beat the current price	LOWER_THAN_CURRENT — amount must exceed current_price (highest_bid.amount, or starting_price if no bids yet).
Amount must clear the minimum increment	MINIMUM_INCREMENT_NOT_MET — amount must be at least current_price + MIN_BID_INCREMENT.
Bidder can't be the seller	SELLER_BIDDING — reject if bidder == auction_lot.auction_event.organizer.
Bidder must not be blocked	BLOCKED_USER — an admin-set flag on the user prevents them from bidding platform-wide.

Why this must be a database transaction

Placing a bid does three things: (1) re-read the lot with a row lock, (2) validate against the checks above, (3) create the Bid and update AuctionLot.highest_bid. All three must happen inside db.transaction.atomic() with select_for_update() on the lot row. Without the row lock, two bidders submitting at almost the same instant can both read the same current_price, both pass validation, and both get recorded as winning — exactly the kind of race condition a POS system worries about with stock, except here it's worse because it's a live competitive feed.

Endpoint

```
POST /api/auctions/lots/{id}/bids
Body: { "amount": 250.00 }

201 Created
{ "id": 91, "amount": "250.00", "status": "ACCEPTED",
  "auction_lot": 7, "created_at": "..." }

422 Unprocessable Entity
{ "status": "REJECTED",
  "rejection_reason": "MINIMUM_INCREMENT_NOT_MET",
  "current_price": "250.00", "minimum_next_bid": "255.00" }
```

Rejected bids are still worth recording (with status REJECTED and a reason) rather than just returning an HTTP error and discarding them — it gives the admin a real audit trail of bidding activity, including attempted abuse.

13. WebSocket System — Real-Time Bidding

This is the module that makes the product feel like a live auction instead of a form that happens to sell things. Django Channels lets the same Django project handle these long-lived connections; groups are the mechanism that makes sure a bid on lot 7 only updates browsers actually watching lot 7.

How Channels groups work here

Each open lot detail page joins a group named lot_{id}. When a bid is accepted, the REST process (or the bid service, wherever the transaction commits) sends the event to that group via the Redis channel layer; Channels delivers it to every consumer subscribed to that group. A bidder who has ever been outbid also has a personal group, user:{id}, used for direct notifications that shouldn't go to everyone watching the lot.

Server → Client events

Event	Group	What it means
bid:new	lot_{id}	A bid was accepted. Payload: full bid object + new current_price. Every open lot page updates the price and prepends the bid to the live feed.
bid:rejected	personal socket only	Sent only to the bidder whose bid failed, with the reason and the minimum next bid, so their UI can show a precise inline error.
outbid	user:{id}	Sent to the previous highest bidder the instant someone outbids them. Frontend shows a toast: "You've been outbid on Lot 7".
lot:closing_soon	lot_{id}	Broadcast when a lot has under 60 seconds left, so the UI can flash the countdown.
lot:ended	lot_{id}	The scheduled job closed this lot. Payload: final status (SOLD/UNSOLD) and the winner, if any.
price:update	all (browse page)	Lightweight event so the public browse page can bump a lot's shown price without opening its detail page.

Client → Server events

Event	What it does
lot:subscribe	Sent when a bidder opens a lot detail page. Server adds the socket to the lot_{id} group and replies with a lot:snapshot (current price, time remaining, last 20 bids).
lot:unsubscribe	Sent when the bidder navigates away. Server removes the socket from the group.

14. Public API — Endpoints + Swagger

The Public API module (Major, 2 points) requires an API key in the x-api-key header, rate limiting, Swagger documentation, and at least one each of GET, POST, PUT, DELETE. DRF gets you most of this almost for free through ViewSets and routers; drf-spectacular generates the Swagger page automatically from those same viewsets.

Method	Endpoint	Description
GET	/api/public/lots	List active/upcoming lots. Query: ?status=, ?category=, ?page=
GET	/api/public/lots/{id}	Get a single lot with current price and bid count
GET	/api/public/lots/{id}/bids	Full bid history for a lot
POST	/api/public/lots	Create a lot under an approved event you organize
PUT	/api/public/lots/{id}	Update a lot (only while SCHEDULED, before it goes ACTIVE)
DELETE	/api/public/lots/{id}	Cancel a lot (organizer or admin only)
GET	/api/public/orders	List your orders as buyer or seller
GET	/api/public/analytics/summary	Revenue, lot count, average final price for a date range

All Public API calls must include the header x-api-key: YOUR_API_KEY, checked by a small piece of custom DRF middleware/permission class. Apply a DRF throttle class (e.g. UserRateThrottle) to every public viewset. Swagger UI will be automatically available at https://localhost/api/docs — use drf-spectacular's SpectacularAPIView and SpectacularSwaggerView, documented with docstrings on each viewset method.

15. Internationalization — Arabic, English, French

Every user-facing string — including bid rejection reasons and status labels — must come from a translation file, not be hardcoded. When the language switches, all text updates instantly without a page reload, including inside an in-progress live bidding page.

Step	Detail
Install	npm install i18next react-i18next i18next-http-backend
Create files	public/locales/en/common.json, public/locales/ar/common.json, public/locales/fr/common.json
Configure	src/i18n.ts — initialize i18next with language detection and fallback to English
Use in components	const { t } = useTranslation(); then t("lot.currentPrice") in JSX
RTL Arabic	When language is "ar", add dir="rtl" to the root element and switch to Tailwind RTL utility classes
Language switcher	A select or button group in the Navbar that calls i18n.changeLanguage("ar")
Backend strings	Error messages returned by the API (rejection reasons, validation errors) should be translatable keys, not hardcoded English — let the frontend map the key to the current language

16. Analytics Dashboard

The analytics dashboard is the seller's and admin's command center. Sellers see numbers scoped to their own auction events; admins see the platform-wide view. It updates in real time via the same WebSocket layer used for bidding — a bid:new event can also bump the relevant chart without a manual refresh.

Chart	What it shows
Revenue by day	Line chart. X-axis = last 30 days. Y-axis = sum of final_price for SOLD lots. Shows trends.
Bids by hour	Bar chart. X-axis = 0–23 hours. Y-axis = bid count. Shows peak bidding times, not just peak sales.
Top 10 lots	Horizontal bar chart. Sorted by final_price or by total bid count. Shows the most contested items.
Sell-through rate	Donut chart. SOLD vs UNSOLD lots — a POS-style stock chart doesn't apply here, this replaces it.
Active bidders	Metric card + small trend line. Unique bidders per day, a proxy for platform engagement.
Today's summary	Metric cards: total revenue, lots closed, average final price, bids placed.

17. 10-Day Execution Plan — Day-by-Day Breakdown

For a Django + React team of five, here is the recommended work division. It replaces a generic P1–P4 split with ownership boundaries that map directly onto the four Django apps already in the repo plus the two halves of the frontend.

Work Division

Person	Main role	Owns
Backend 1	Core backend / Auth / Tech Lead	Django + Docker + Nginx setup, Redis + Channels infra, JWT auth, 2FA, roles & permissions, deploy sanity
Backend 2	Catalog + Auctions	categories, products, inventory; auction_events (draft → submit → approve/reject); auction_lots
Backend 3	Bidding & Sales Engine	bids (validation service, rejection reasons), lot auto-closing job, orders, payments, analytics endpoints
Frontend 1	Bidder UI	Browse page, Lot Detail + live bidding panel, WebSocket hook, My Bids / My Wins, checkout & payment flow
Frontend 2	Admin / Seller UI	Login/register, My Auctions (create/submit/add lots), Admin approval queue, Analytics dashboard, role-based nav, Privacy/ToS

The team is not starting from zero: the Django models for accounts, products, auctions, and orders already exist, and Docker/Nginx/HTTPS are already wired up. That head start is exactly what makes 10 focused days realistic instead of the original 8 weeks. Each day's Goal is a concrete test — if you can't pass it, you are not done, and the next day's plan assumes you are.

Day 1 — Foundation finalization

Backend 1: Add redis + websocket services to docker-compose.yml. Verify nginx.conf's existing /ws/ → :8080 route end-to-end with a dummy echo consumer. Install DRF, simplejwt, corsheaders. Build register/login/logout/me endpoints with JWT httpOnly cookies.

Backend 2: Review the products/auctions models as a team. Finalize Category/Product/Inventory serializer skeletons. Write the team's seed script: 1 admin, 2 sellers, 3 bidders, 3 categories, 6 products, 1 approved auction event with 3 lots at different statuses.

Backend 3: Review the orders/payments/bids models as a team. Sketch the bid validation service interface (function signatures, inputs/outputs) so everyone can code against it before it's built.

Frontend 1: Vite + React + TypeScript + Tailwind scaffold in a new frontend/ folder. Install dependencies, base routing skeleton, page layout shell.

Frontend 2: Set up React Query + Zustand config. Base component library: Button, Input, Modal, Badge. Tailwind theme tokens (colors, spacing).

Goal: docker compose up shows 6 green containers (db, redis, backend, websocket, frontend, nginx). https://localhost loads the Nginx default/React placeholder page. Postman can register and log in against the real DB.

Day 2 — Auth complete + frontend shell

Backend 1: Role-based DRF permission classes (IsAdmin, IsAuctionOwner). 2FA setup/verify/confirm endpoints with pyotp.

Backend 2: Write auction_events serializers and CRUD views (draft/submit).

Backend 3: Build the admin approve/reject endpoint with rejection_reason, working closely with Backend 2 on the AuctionEvent state machine.

Frontend 1: Login/Register pages, Zustand auth store, Axios client with cookie credentials, protected route wrapper.

Frontend 2: RequireRole wrapper. Base Navbar/Sidebar shell with role-based links (empty states for now).

Goal: A bidder can log in via the browser, land on a placeholder Browse page, and get redirected if not logged in. Admin flag is visible in the JWT payload / /me response.

Day 3 — Auction events end-to-end

Backend 2: Finalize auction_events list/detail/update endpoints, enforcing organizer_can_edit / editing_deadline server-side.

Backend 3: Refine the admin approval queue endpoint; add a rejection-reason validation path.

Backend 1: Review Day 1–2 auth and event PRs. Confirm HTTPS + cookie flags (Secure, SameSite) work correctly through Nginx.

Frontend 2: "My Auctions" page (create/submit form, status badge) and the Admin approval queue page (reason textarea).

Frontend 1: Wire both pages to the real API with React Query; loading/error states.

Goal: A seller can create and submit an auction event; an admin can approve or reject it with a reason; status changes persist and are visible in both UIs.

Day 4 — Auction lots

Backend 2: Lot CRUD endpoints scoped to the organizer's own approved events. Enforce the unique_lot_number_per_event constraint at the API layer with a clear error.

Backend 3: Finalize Order/Payment fields needed downstream; sketch the lot-closing job structure for Day 7.

Backend 1: Add product image upload support (or a placeholder image field) for lot cards.

Frontend 1: Public Browse page (grid of lot cards) and Lot Detail page (static, no live bidding yet) with a countdown component.

Frontend 2: Seller-side "Add Lot" form inside an event's page.

Goal: A seller can add lots to an approved event; the public Browse page lists live and upcoming lots with correct starting prices and countdowns.

Day 5 — Bidding engine

Backend 3: Bid validation service (services.py) with select_for_update() and all five checks from Section 12. POST /bids endpoint.

Backend 2: Bid history endpoint per lot, ordered newest-first, exposed alongside the lot detail endpoint.

Backend 1: Write a race-condition test: fire two near-simultaneous bids at the same lot and confirm only one wins.

Frontend 1: Bid placement panel on Lot Detail: amount input, minimum-next-bid hint, inline error display for each rejection reason.

Frontend 2: Reflect current price and bid count on Browse page cards.

Goal: A bid placed via Postman updates AuctionLot.highest_bid correctly, and invalid bids return the exact rejection reason from Section 12.

Day 6 — Real-time bidding

Backend 1 + Backend 3: Channels consumer (consumers.py), routing.py, lot_{id} groups, and user:{id} groups. Broadcast bid:new, bid:rejected, outbid, lot:closing_soon.

Backend 2: Wire a price:update broadcast into the lot service layer so the Browse page can stay live too.

Frontend 1: useAuctionSocket hook: connect on Lot Detail mount, disconnect on unmount, update the bid store on each event, live countdown sync.

Frontend 2: Toast/notification component for outbid and auction-won alerts.

Goal: Two browser tabs open on the same lot see a new bid, the updated price, and an outbid toast appear instantly in both, with no refresh.

Day 7 — Closing lots, orders, payments

Backend 3: Scheduled closing job (Celery Beat or a lightweight loop): finds ACTIVE lots past end_time, sets SOLD/UNSOLD, creates the Order, broadcasts lot:ended. Must be idempotent.

Backend 2: Order list/detail endpoints for both the buyer and seller views.

Backend 1: Payment endpoint: creates a Payment record, simulates success/failure, updates Order.status. Verify the closing job survives a container restart without double-processing a lot.

Frontend 1: "My Wins" page and the checkout/payment page.

Frontend 2: Order status page with a status timeline, for both buyer and seller.

Goal: When a lot's end_time passes, the winning bidder automatically gets an Order; a mock payment moves it to PAID; the seller sees the sale in their orders list.

Day 8 — Analytics + Public API

Backend 3: Analytics endpoints: revenue by day, bids by hour, top lots, sell-through rate, active bidders.

Backend 2: Public API viewsets for lots and orders (GET/POST/PUT/DELETE) + x-api-key permission class.

Backend 1: DRF throttling classes, drf-spectacular setup, /api/docs page — verify all 8+ endpoints appear with correct methods.

Frontend 2: Analytics dashboard page with Recharts, date range picker, CSV export button for the current view.

Frontend 1: General bug pass on Browse/Lot Detail; small UX polish (trending badge, empty states).

Goal: Charts render real database data with a date range filter. curl -H "x-api-key: KEY" https://localhost/api/public/lots returns JSON. /api/docs shows Swagger UI.

Day 9 — i18n, 2FA, export/import, static pages

Backend 1: Translatable error-message keys for API responses (rejection reasons, validation errors).

Backend 2: CSV bulk-import endpoint for lots.

Backend 3: CSV export endpoints for bid history and orders.

Frontend 1: i18next wiring across bidder-facing pages, language switcher, RTL layout for Browse/Lot Detail/checkout.

Frontend 2: 2FA frontend flow (QR code, code entry, login-time TOTP prompt), Privacy Policy / Terms of Service pages, RTL for admin/seller pages.

Goal: Full Arabic UI with RTL layout works end-to-end. 2FA setup-and-login works with a real authenticator app. CSV export/import both work. Privacy Policy and Terms of Service are live and linked from the footer.

Day 10 — Polish, README, rehearsal

All: Open DevTools on every page as USER and ADMIN, fix every red error and yellow warning, check the Network tab for failed requests.

All: Write the full README.md (Section 21). Confirm HTTPS end-to-end and multi-user simultaneous bidding one more time.

All: Two full evaluation rehearsals: each person presents their part out loud; practice live code changes on request.

Backend 1: Final docker compose up --build from a clean clone to catch any environment-specific bug before evaluation.

Goal: Zero console errors in Chrome across every page for every role. README is complete. Every person can explain their code and make a small live change confidently.

18. Team Roles and Responsibilities

These roles must be documented in your README.md. During evaluation, every team member will be asked to explain how roles were distributed and how you coordinated. The Work Division table in Section 17 defines who owns which code; the roles below are the project-management layer on top of that — with a 5-person team, Backend 1 doubles as Tech Lead since that person already owns the infra, and the team should assign Product Owner and Scrum Master among the remaining four (either fixed for the whole 10 days or rotated every few days — agree on this on Day 1).

Product Owner (PO)

- Owns the product backlog — the ordered list of what to build next
- Makes final decisions when the team disagrees on features (e.g. minimum bid increment value, whether sellers can bid at all)
- Validates completed features against requirements
- Ensures Privacy Policy and Terms of Service are adequate for a site handling real money flows
- Speaks for the project during evaluation introductions

Project Manager / Scrum Master

- Runs daily 15-minute stand-ups given the compressed 10-day timeline
- Maintains the GitHub Issues board — every task is an Issue
- Tracks who is working on what and when it will be done
- Identifies blockers and escalates immediately — a 10-day plan has no slack for a day lost to confusion

Tech Lead / Architect

- Owns Docker, Nginx, and the redis/websocket service split described in Section 9
- Owns the Channels consumer and group design in Section 13
- Defines API conventions (response format, error format, naming) across all four Django apps
- Reviews all Pull Requests for the bidding engine and the auction lifecycle — the two riskiest pieces of code

Developers (all members)

- Implements assigned features according to the day-by-day plan
- Opens a Pull Request for every feature — never pushes directly to main
- Writes at least one other team member's PR review per day given the compressed schedule
- Must be able to explain every line they wrote during evaluation

19. Git Workflow and Commit Rules

The evaluators will look at your Git history. They need to see commits from ALL team members with clear, meaningful messages. A history of "fix", "update", "asdf" will raise red flags — this rule doesn't change with the pivot.

Branch strategy

Branch	Purpose
main	Always deployable. Only merge via PR. Never push directly to main.
develop	Integration branch. Merge your feature branches here first for testing.
feature/bidding-engine	One branch per feature. Format: feature/short-description.
feature/auction-lifecycle	Examples: feature/auction-lifecycle, feature/websocket-bids, feature/analytics
fix/bid-race-condition	Bug fixes. Format: fix/description-of-bug

Good vs bad commit messages

- GOOD — Add minimum-increment validation to bid service
- GOOD — Implement row-locked transaction for concurrent bid placement
- GOOD — Add Channels consumer for lot_{id} bid broadcasts
- GOOD — Add Arabic RTL support with i18next and Tailwind RTL plugin
- BAD — fix / update stuff / wip / asdf / changes

20. Mandatory Checklist — Rejection Criteria

Check every item below before submission. A single unchecked item can result in automatic project rejection regardless of how good the rest of the project is.

General Requirements

- Web application with frontend, backend, AND database — all three exist
- Entire application starts with a single command: docker compose up --build
- Git repository has commits from ALL team members
- All commit messages are clear and meaningful
- Application runs correctly in the latest stable version of Google Chrome
- Zero errors and zero warnings in the browser console on every page, for every role
- Multi-user support: two or more bidders competing on the same lot at once causes no errors and no lost bids
- All browser-to-server communication uses HTTPS, including the WebSocket connection (wss://)

Pages and Content

- Privacy Policy page exists with real, relevant content
- Terms of Service page exists with real, relevant content, including how winning bids and payment obligations work
- Both pages are accessible without logging in and linked from the footer on every page

Technical Requirements

- Frontend is responsive — works correctly on both mobile and desktop
- A CSS framework is used (Tailwind)
- .env is gitignored; .env.example is committed with all required variable names
- Database has a clear schema — evaluator will ask to see it
- User passwords are hashed — never stored in plain text
- All form inputs are validated on the frontend AND the backend (DRF serializers)

Module Requirements

- Framework module: Django REST Framework backend + React frontend both genuinely using framework features
- WebSocket module: live bid updates working in a real demo for the evaluator, across two browser tabs
- User management module: profile page, avatar, online status visible
- Permissions module: USER vs ADMIN genuinely have different access; a seller genuinely cannot bid on their own lot
- Analytics module: charts show real database data with a date range filter
- Public API module: /api/docs accessible, x-api-key required, rate limiting active
- ORM module: Django ORM used for all database access — no raw SQL queries
- 2FA module: can demonstrate full setup and login flow with an authenticator app
- Multi-language module: 3 languages working with all UI strings translated, including bid rejection reasons
- RTL module: Arabic layout fully mirrors (sidebar, text direction, form alignment, live bid feed)
- Data export module: CSV export works and downloads a valid file

21. README.md Requirements

The README is evaluated as seriously as the code. Write it progressively across the 10 days, not on the last day — with a 10-day sprint there is no slack to write it all at once at the end.

Section	Required content
Line 1 (italicized)	*This project has been created as part of the 42 curriculum by login1, login2, login3, login4.*
## Description	Project name, what it does, key features list, why you pivoted from a POS idea to a live auction platform.
## Team Information	For each member: name/login, assigned role(s), responsibilities description.
## Project Management	How you organized work (GitHub Issues), meeting schedule, communication channel, task distribution across the 10 days.
## Technical Stack	Django + DRF + Channels backend + why. React frontend + why. PostgreSQL + Redis + why. Other libraries + why.
## Database Schema	List all 9 tables with their key fields and relationships. Include the models.py excerpts or an ERD image.
## Instructions	Prerequisites (Docker Desktop, Git). Step-by-step: clone, copy .env.example to .env, fill values, run docker compose up --build, access https://localhost.
## Features List	Every feature with: description, which module it satisfies, which team member(s) built it.
## Modules	Every module claimed: name, type (Major/Minor), points, implementation description, which member built it.
## Individual Contributions	Per person: list every feature, module, and component they built. Be specific. Evaluators will cross-reference this with git log.
## Resources	Links to documentation, tutorials, and articles used. Description of how AI tools were used, for which tasks.

You have everything you need

This document contains the complete plan: what to build, what to learn, how to build it, and when to build it. The backend foundation already exists in the repository. The only thing left is to start. Day 1 action: Backend 1 adds Redis and the websocket service to docker-compose.yml while Frontend 1 and Frontend 2 scaffold the React app. Everyone else builds on the models that are already there. From that moment, you are building.